

Algorytmy Optymalizacji Dyskretnej

Rafał Wochna 279752

1 Opis implementacji algorytmów i ich złożoności

1.1 Algorytm Dijkstry

Algorytm Dijkstry został zaimplementowany z wykorzystaniem kolejki priorytetowej (`std::priority_queue`) w celu efektywnego wybierania wierzchołka o najmniejszej odległości. Dla każdego wierzchołka przechowywana jest minimalna odległość od źródła (`dist`). Jeśli odległość do sąsiedniego wierzchołka może zostać zaktualizowana, wierzchołek ten jest dodawany do kolejki priorytetowej.

Złożoność czasowa:

- Operacja dodawania do kolejki priorytetowej (`push`) oraz usuwania elementu o najmniejszym priorytecie (`pop`) ma złożoność $O(\log V)$, gdzie V to liczba wierzchołków.
- Przejście przez wszystkie krawędzie grafu ma złożoność $O(E)$, gdzie E to liczba krawędzi.

Łączna złożoność czasowa wynosi $O((V + E) \log V)$.

1.2 Algorytm Diala

Algorytm Diala został zaimplementowany z wykorzystaniem struktury kubełków (`std::deque`), które przechowują wierzchołki o określonych odległościach. Liczba kubełków wynosi $C + 1$, gdzie C to maksymalna waga krawędzi w grafie. Wierzchołki są przetwarzane w kolejności rosnących odległości, co eliminuje potrzebę korzystania z kolejki priorytetowej.

Złożoność czasowa:

- Każdy wierzchołek może być przetworzony maksymalnie C razy, co daje złożoność $O(V \cdot C)$, gdzie V to liczba wierzchołków, a C to maksymalna waga krawędzi.
- Przejście przez wszystkie krawędzie grafu ma złożoność $O(E)$, gdzie E to liczba krawędzi.

Łączna złożoność czasowa wynosi:

$$O(V \cdot C + E)$$

1.3 Algorytm Radix Heap

Algorytm Radix Heap wykorzystuje strukturę danych **RadixHeap**, która pozwala na efektywne zarządzanie wierzchołkami w kolejce priorytetowej. Struktura ta grupuje wierzchołki w kubelki na podstawie wartości ich odległości, co pozwala na szybkie znajdowanie minimalnych wartości.

Złożoność czasowa:

- Operacje **push** i **pop** w strukturze **RadixHeap** mają złożoność $O(\log C)$, gdzie C to maksymalna różnica wartości w kubelkach.
- W przypadku 64-bitowych liczb całkowitych, $\log C \leq 64$, co oznacza, że operacje te mają w praktyce stałą złożoność $O(1)$.
- Przejście przez wszystkie krawędzie ma złożoność $O(E)$.

Łączna złożoność czasowa wynosi:

$$O(V + E)$$

1.4 Podsumowanie

Algorytm	Złożoność czasowa
Dijkstra	$O((V + E) \log V)$
Dial	$O(V \cdot C + E)$
Radix Heap Dijkstra	$O(V + E)$

Tabela 1: Porównanie złożoności czasowych zaimplementowanych algorytmów.

Tabela 2: Porównanie czasów wykonania algorytmów

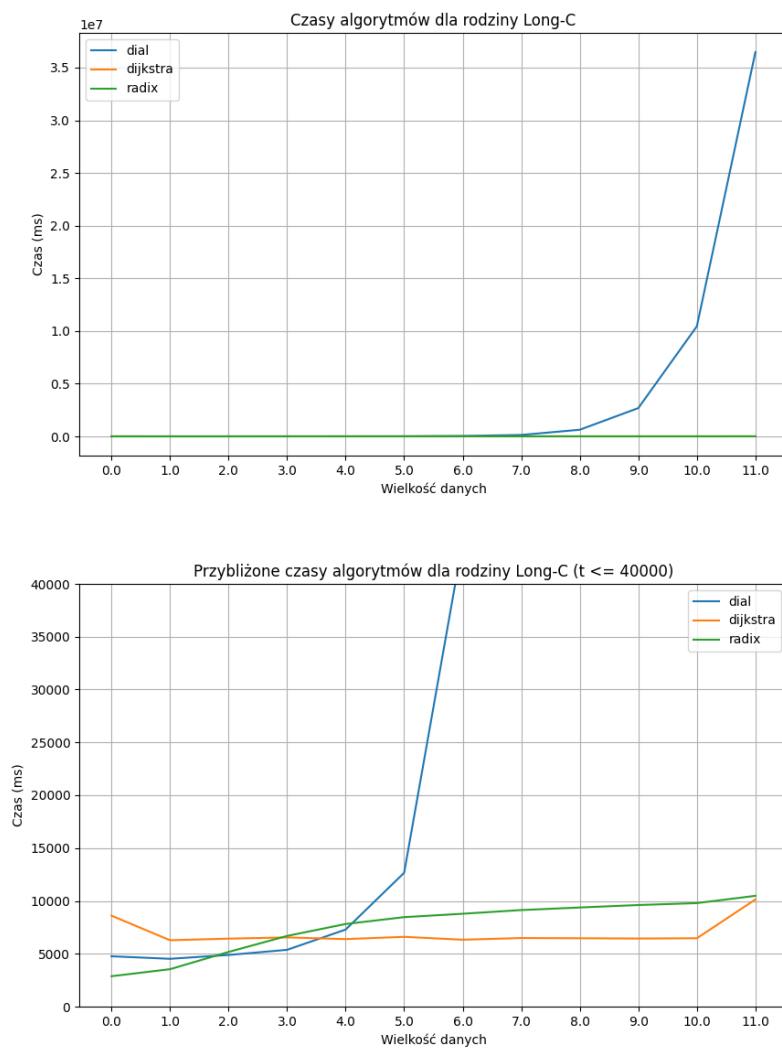
Nazwa pliku	Dial	Dijkstra	Radix
Long-C.11.0	89409.80	35.49	45.66
Long-C.15.0.1	N/A	2.91	5.53
Long-n.21.0	1769320.00	455.41	563.24
Random4-C.11.0	3267.46	208.69	188.56
Random4-C.15.0	N/A	202.85	193.79
Random4-n.21.0	3435.51	884.75	684.27
Square-C.11.0	10211.10	97.17	104.43
Square-C.15.0.1	N/A	0.16	0.25
Square-n.21.0	43095.30	420.14	434.19
USA-road-d.USA	10833.10	6434.44	5601.00

Tabela 3: Porównanie pierwszych danych z plików

Rodzina	Algorytm	Źródło	Cel	Wartość
Long-C.11.0	Dial	1	32768	6830134480
Long-C.11.0	Dijkstra	1	32768	6830134480
Long-C.11.0	Radix	1	32768	6830134480
Long-C.15.0.1	Dial	N/A	N/A	N/A
Long-C.15.0.1	Dijkstra	1	32768	346117263789
Long-C.15.0.1	Radix	1	32768	346117263789
Long-n.21.0	Dial	1	2097152	31336751771
Long-n.21.0	Dijkstra	1	2097152	31336751771
Long-n.21.0	Radix	1	2097152	31336751771
Random4-C.11.0	Dial	1	32768	15329690
Random4-C.11.0	Dijkstra	1	32768	15329690
Random4-C.11.0	Radix	1	32768	15329690
Random4-C.15.0	Dial	N/A	N/A	N/A
Random4-C.15.0	Dijkstra	1	1048576	3471241820
Random4-C.15.0	Radix	1	1048576	3471241820
Random4-n.21.0	Dial	1	2097152	9051281
Random4-n.21.0	Dijkstra	1	2097152	9051281
Random4-n.21.0	Radix	1	2097152	9051281
Square-C.11.0	Dial	1	32761	621678561
Square-C.11.0	Dijkstra	1	32761	621678561
Square-C.11.0	Radix	1	32761	621678561
Square-C.15.0.1	Dial	N/A	N/A	N/A
Square-C.15.0.1	Dijkstra	1	32761	3673878785
Square-C.15.0.1	Radix	1	32761	3673878785
Square-n.21.0	Dial	1	2096704	714640488
Square-n.21.0	Dijkstra	1	2096704	714640488
Square-n.21.0	Radix	1	2096704	714640488
USA-road-d.USA	Dial	1	23947347	23228284
USA-road-d.USA	Dijkstra	1	23947347	23228284
USA-road-d.USA	Radix	1	23947347	23228284

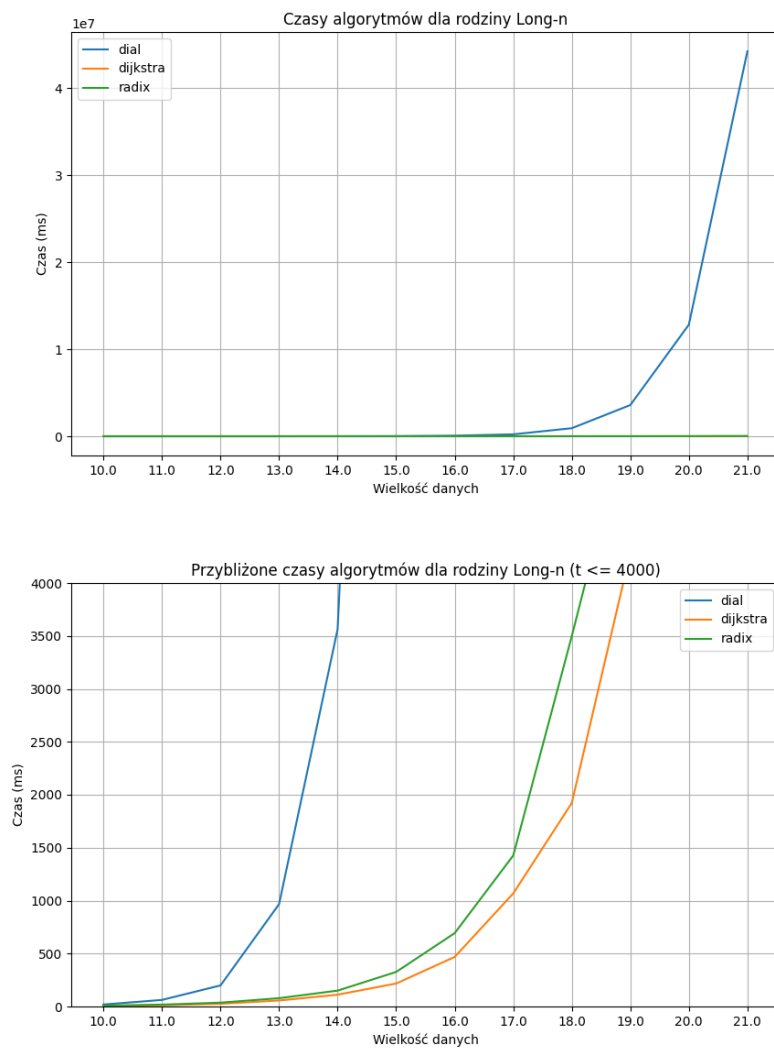
Wykresy czasów algorytmów

Rodzina Long-C



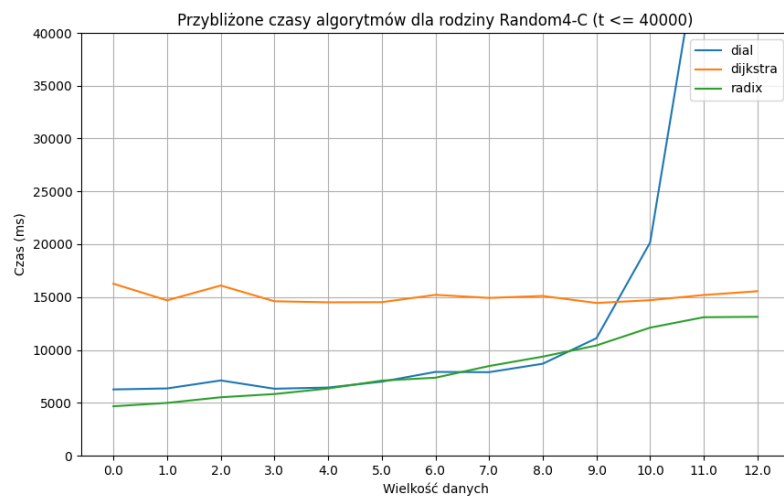
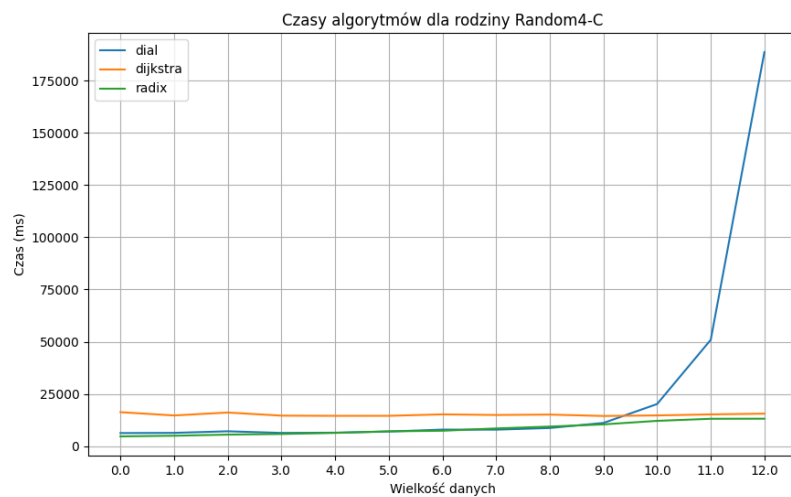
Rysunek 1: Wykresy czasów algorytmów dla rodziny Long-C

Rodzina Long-n



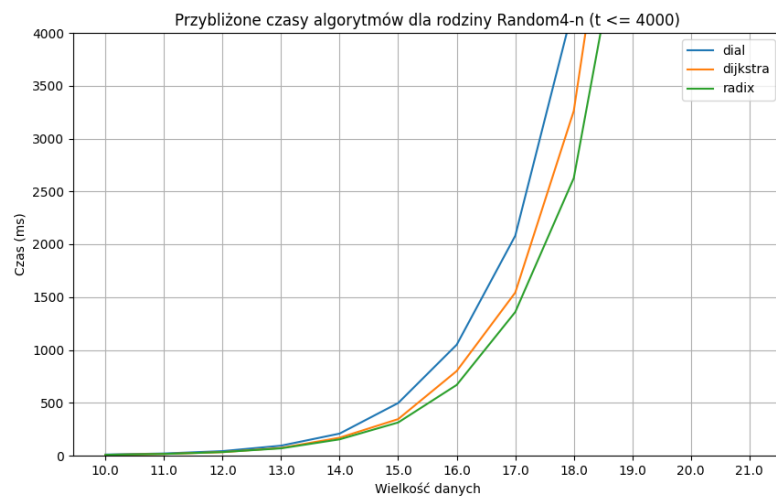
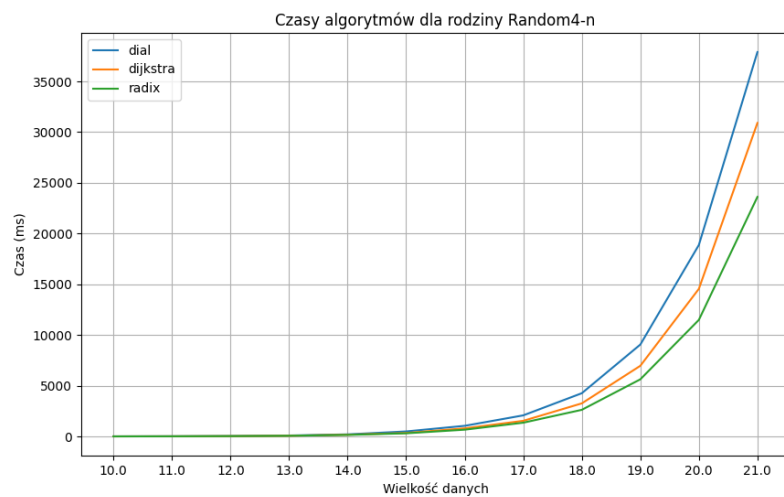
Rysunek 2: Wykresy czasów algorytmów dla rodziny Long-n

Rodzina Random4-C



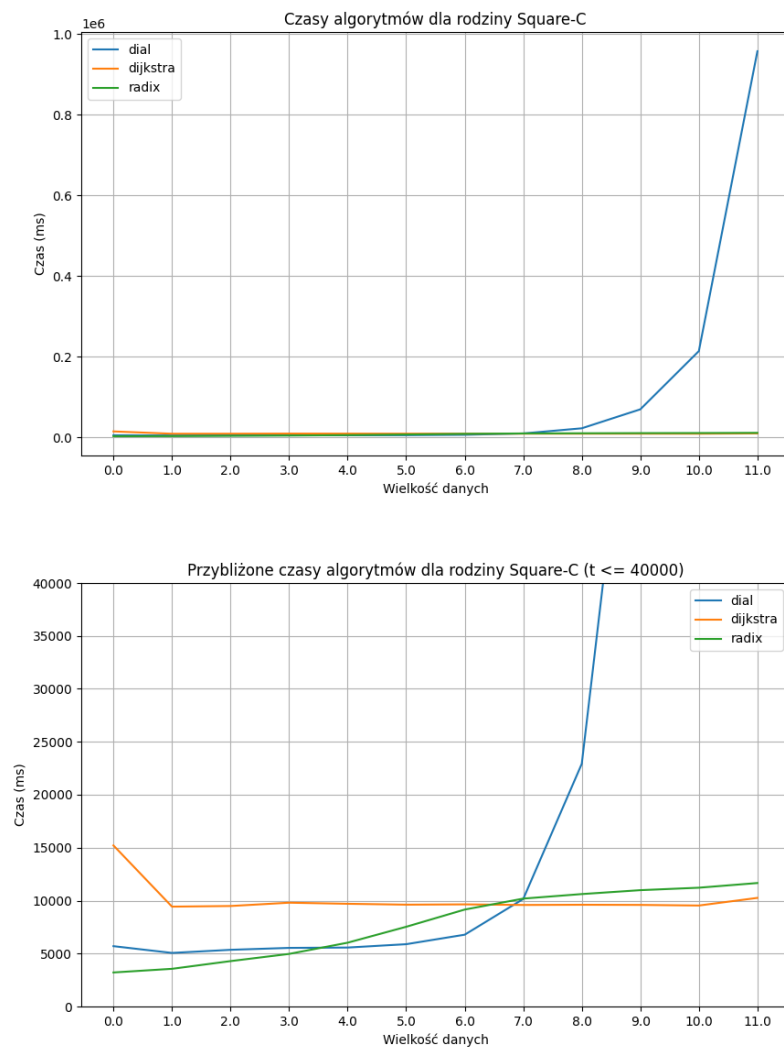
Rysunek 3: Wykresy czasów algorytmów dla rodziny Random4-C

Rodzina Random4-n



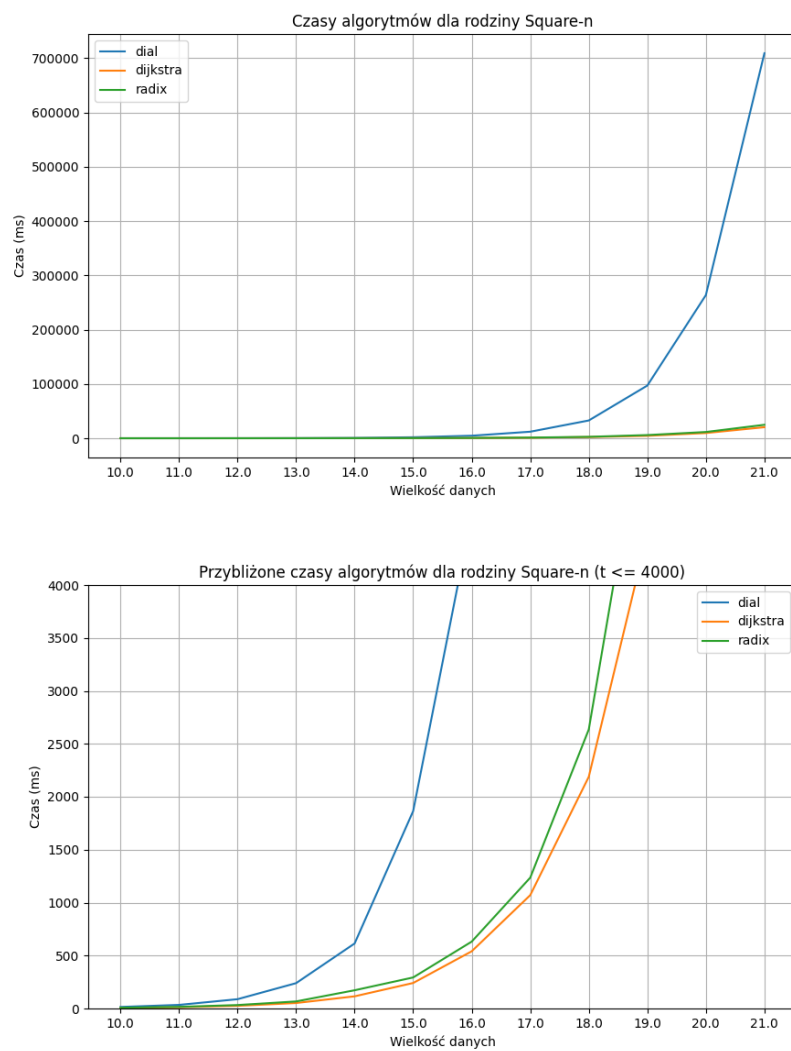
Rysunek 4: Wykresy czasów algorytmów dla rodziny Random4-n

Rodzina Square-C



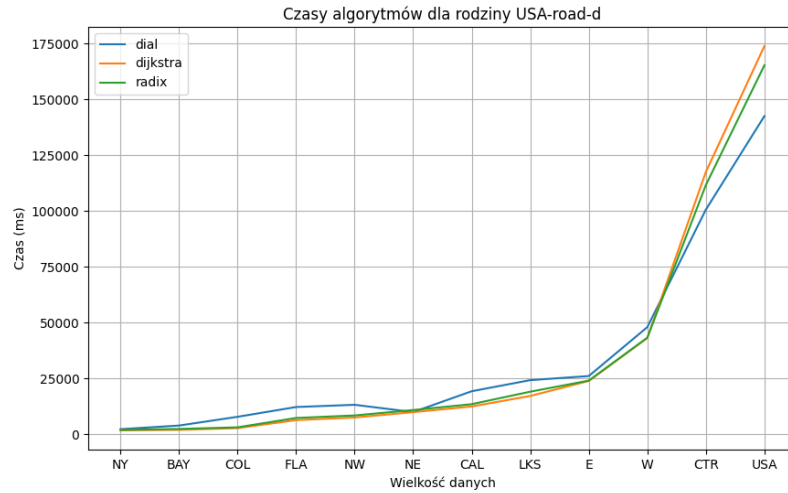
Rysunek 5: Wykresy czasów algorytmów dla rodziny Square-C

Rodzina Square-n



Rysunek 6: Wykresy czasów algorytmów dla rodziny Square-n

Rodzina USA-road-d



Rysunek 7: Wykres czasów algorytmów dla rodziny USA-road-d

2 Obserwacje i wnioski

Algorytmy Dijkstry i Radix Heap wykazują porównywalną wydajność we wszystkich rodzinach danych. Ich działanie jest niemal niezależne od wartości (C) (największego kosztu krawędzi w grafie), co sprawia, że są stabilne i efektywne nawet przy dużych wartościach (C) .

Z kolei algorytm Diala wyróżnia się bardzo dobrą wydajnością dla małych wartości (C) oraz (n) (liczby wierzchołków). W takich przypadkach działa wyjątkowo szybko porównywalnie do algorytmów Dijkstry i Radix Heap. Algorytm Diala przestaje działać dla dużych wartości $(C > 11)$. Wynika to z faktu, że jego złożoność asymptotyczna $(O(E + V \cdot C))$ staje się niepraktyczna dla dużych C , ponieważ liczba operacji rośnie liniowo wraz ze wzrostem C . W takich przypadkach algorytm nie jest w stanie efektywnie przetwarzać danych w rozsądnym czasie.